# Product Requirements Document (PRD): NaviLite

Klien Manajemen Database Desktop Native Ringan Berbasis C# (.NET 8) + Rust Core dengan Driver MySQL & Mesin Data Transfer Bergaya SQLyog

| Atribut Dokumen | Nilai / Spesifikasi | Atribut Teknis | Target Arsitektur |
|---|---|---|---|
| **Nama Proyek** | NaviLite (Lightweight Navicat Clone) | **UI Framework** | C# / .NET 8 Native Desktop (Avalonia UI / WinUI 3) |
| **Versi Dokumen** | v1.0 — Spesifikasi MVP | **Backend / Core Engine** | Rust (`cdylib`, `tokio`, `mysql_async`, `russh`) |
| **Tanggal Rilis PRD** | 16 Oktober 2026 | **Jembatan Interop** | C-ABI FFI via `csbindgen` & Pinned Buffers |
| **Driver Database Awal** | MySQL & MariaDB (TCP/IP, SSL, SSH Tunnel) | **Fitur Unggulan** | Cross-Connection Data Transfer (Mekanisme SQLyog) |

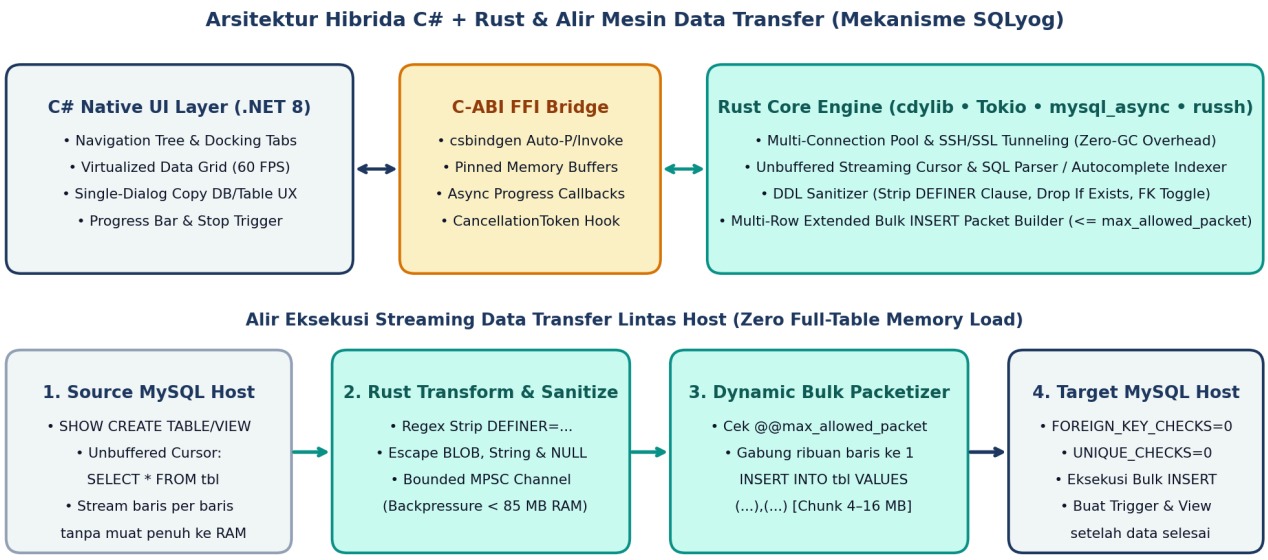
## 1. Visi Produk: Memadukan Ergonomi Navicat dengan Kecepatan Eksekusi SQLyog

Pengembang perangkat lunak dan administrator basis data (DBA) membutuhkan perkakas harian yang instan saat dibuka, hemat memori, namun andal saat memigrasikan data berukuran besar antar-server. Navicat Premium unggul dalam tata letak kerja yang terstruktur (*Navigation Pane*, *Data Grid*, dan *Object Designer*), tetapi versi penuhnya memuat banyak modul berat seperti *Business Intelligence (BI)*, *Data Vault Modeling*, dan *AI Chat* yang meningkatkan jejak memori. Sebaliknya, SQLyog sangat disukai karena fitur **Copy Database/Table to Different Host/DB** yang mampu menyalin skema dan jutaan baris data lintas server dalam satu jendela dialog tanpa menguras RAM.

**NaviLite** memadukan keunggulan keduanya ke dalam arsitektur desktop *native* modern: antarmuka bersih bergaya Navicat yang dibangun menggunakan **C# (.NET 8)** di atas mesin pemrosesan asinkron **Rust**. Fokus MVP v1.0 adalah menghadirkan dukungan penuh untuk **MySQL / MariaDB** beserta fitur **Data Transfer antar-koneksi berkecepatan tinggi** yang mengadopsi alur kerja dan mekanisme *streaming bulk packet* milik SQLyog.

## 2. Arsitektur Hibrida C# + Rust & Desain Interoperabilitas FFI

Pemisahan tegas antara lapisan presentasi (C#) dan mesin protokol database (Rust) menjamin *UI thread* tetap responsif pada 60 FPS meskipun aplikasi sedang menyalin tabel berisi puluhan juta baris atau mengeksekusi skrip SQL besar di latar belakang.




Gambar 1: Arsitektur Hibrida C# (.NET 8) ↔ C-ABI FFI (csbindgen) ↔ Rust Core Engine serta Alir Pipeline Streaming Data Transfer Lintas Host.

| Lapisan Arsitektur | Teknologi & Pustaka Utama | Tanggung Jawab Teknis & Mekanisme Optimasi |
|---|---|---|
| **Presentation & UI Layer** | C# 12, .NET 8, Avalonia UI (Lintas Platform) atau WinUI 3 / WPF (Windows Native), MVVM Toolkit | Merender pohon navigasi skema, *tabbed workspace*, *Virtualized Data Grid* (hanya merender ~50 baris yang tampak di layar), editor SQL dengan *syntax highlighting*, serta dialog *Copy Database to Different Host*. |
| **Interop & Memory Bridge** | `csbindgen` (Rust C-ABI FFI Generator), `LibraryImport`, `ReadOnlySpan<byte>` | Menghasilkan *binding* P/Invoke C# otomatis saat kompilasi Cargo. Pertukaran data hasil kueri menggunakan blok memori *pinned* berformat biner ringkas (panjang kolom + *byte payload*) untuk menghindari alokasi objek berlebih dan jeda *Garbage Collector* (GC). |
| **Database & Transfer Core** | Rust (`cdylib`), `tokio` (Multi-thread Async Runtime), `mysql_async`, `russh`, `keyring` | Mengelola *connection pool* MySQL/MariaDB, *SSH port forwarding* di memori tanpa proses eksternal, *unbuffered cursor streaming*, sanitasi DDL (penghapusan klausa `DEFINER`), serta pembuat paket *extended bulk INSERT*. |

## 3. Cakupan Fitur MVP v1.0 & Pemangkasan Fitur Berat

Untuk menjaga ukuran aplikasi di bawah 25 MB dan pemakaian RAM *idle* di bawah 45 MB, NaviLite memangkas fitur-fitur sekunder dari Navicat Premium dan memusatkan pengembangan pada modul esensial berikut:

| Kategori Modul | Fitur yang Disertakan pada NaviLite MVP v1.0 | Fitur Navicat yang Dipangkas (Out of Scope) |
|---|---|---|
| **Manajemen Koneksi & Keamanan** | Koneksi MySQL & MariaDB via TCP/IP, Socket, SSL/TLS, dan SSH Tunnel; pengelompokan warna koneksi (Merah/Kuning/Hijau); penyimpanan sandi terenkripsi di OS Keychain; impor/ekspor koneksi. | Navicat Cloud Sync, On-Prem Server, Kerberos/LDAP/PAM kompleks, serta driver NoSQL/Snowflake (ditunda ke v2.0). |
| **Navigasi & Desain Objek** | *Navigation Tree lazy-load* (*Tables*, *Views*, *Procedures*, *Functions*, *Triggers*, *Events*); pencarian objek instan; *Table Designer* visual (Kolom, Indeks, *Foreign Key*, Trigger, tab pratinjau DDL). | Model Workspace (Conceptual, Logical, Data Vault 2.0, Dimensional Modeling) dan Reverse/Forward Engineering berat. |
| **Data Viewer & Editor** | *Virtualized Grid View* & *Form View*; *Batch Editing* dengan bilah konfirmasi *Apply/Discard*; penyunting sel *Text/Hex/JSON/NULL*; bilah filter `WHERE` & `ORDER BY` cepat; *Copy As INSERT/UPDATE/CSV*. | Data Profiling statistik berat, BFile Viewer khusus Oracle, dan kustomisasi warna per tipe data. |
| **SQL Query Editor** | *Syntax highlighting* SQL cepat; *autocomplete* cerdas berbasis *cache* skema aktif; eksekusi seluruh skrip (`Ctrl+F9`) atau pernyataan di kursor (`F9`); sematkan hasil kueri (*Pin Result Tab*); tab *EXPLAIN*. | Visual Query Builder drag-and-drop, PL/SQL Debugger, dan Multi-Model AI Chat Room. |
| **Migrasi, Transfer & Backup** | **Data Transfer Antar-Koneksi (Mekanisme SQLyog)**; *Streamed SQL Dump* (Struktur / Data); *Execute SQL Script* berukuran gigabita tanpa muat ke RAM; Impor/Ekspor CSV & JSON cepat. | Data Generation (30+ generator sintetis), Data Masking, Data Dictionary PDF Builder, BI Charts, dan Job Scheduler. |

### Struktur Hierarki Menu Bar & Pintasan Keyboard Utama

- **Menu File**: *New Connection* (`Ctrl+N`), *Open SQL File...* (`Ctrl+O`), *Save SQL File* (`Ctrl+S`), *Import/Export Connections*, *Exit* (`Alt+F4`).
- **Menu Edit**: *Undo/Redo*, *Copy Rows As* (`INSERT`, `UPDATE`, `CSV`, `TSV`, `JSON`), *Find in Grid/Editor* (`Ctrl+F`), *Replace* (`Ctrl+H`).
- **Menu View**: *Navigation Pane* (`Alt+1`), *DDL & History Log Pane* (`Alt+2`), *Toggle Grid / Form View* (`Ctrl+G`), *Focus Mode* (`F11`), *Theme (Dark / Light)*.
- **Menu Database**: *Create Database...*, *Alter Database...*, *Drop Database*, **Copy Database to Different Host/Database...** (`Ctrl+Shift+C`), *Backup/Dump Database as SQL...* (`Ctrl+Shift+D`), *Execute SQL Script...* (`Ctrl+Shift+E`).
- **Menu Table**: *Open Table*, *Design Table* (`Ctrl+D`), *Create Table*, **Copy Table(s) to Different Host/Database...**, *Import CSV/JSON...*, *Export Table Data...*, *Truncate Table*, *Drop Table*.
- **Menu Tools**: *Server Process List* (memantau dan menghentikan kueri melalui `SHOW FULL PROCESSLIST` & `KILL`), *User & Privilege Manager*, *Flush Server Objects* (Privileges/Tables/Logs), *Options*.

## 4. Spesifikasi Mendalam: Fitur Data Transfer Antar-Koneksi (Mekanisme SQLyog)

Fitur **Copy Database/Table to Different Host/Database** adalah pembeda utama NaviLite dibandingkan klien database ringan lainnya. Alih-alih menggunakan *wizard* multi-halaman yang lambat, fitur ini mereplikasi alur kerja satu dialog dan arsitektur transfer paket milik SQLyog.

### A. Spesifikasi Antarmuka & Opsi Konfigurasi Dialog (Single-Dialog UX)

Dialog dibuka melalui klik kanan pada nama Database atau Tabel di *Navigation Pane* → **Copy Database/Table to Different Host/Database...**. Seluruh konfigurasi berada dalam satu layar tunggal:

- **Panel Sumber (Kiri)**: Menampilkan label koneksi dan database sumber beserta pohon objek yang memiliki kotak centang (*checkbox*) per kategori: `Tables` (dengan tampilan jumlah baris estimasi dan ukuran MB), `Views`, `Stored Procedures`, `Functions`, `Triggers`, dan `Events`. Tersedia kotak filter cepat dan tombol *Select All / Deselect All*.
- **Panel Tujuan & Opsi Penyalinan (Kanan)**:
  - **Target Connection & Target Database**: *Dropdown* untuk memilih koneksi tujuan yang terdaftar (termasuk koneksi yang melewati SSH Tunnel berbeda) dan *editable combobox* untuk memilih database tujuan atau mengetikkan nama database baru (otomatis menjalankan `CREATE DATABASE IF NOT EXISTS` dengan *charset/collation* yang disamakan dengan sumber).
  - **Mode Penyalinan**: Pilihan tombol radio antara **Structure and Data** (default) atau **Structure Only** (hanya menyalin DDL tanpa baris data).
  - **Drop if exists in target** (Checkbox — Default: Aktif): Menghapus objek bernama sama di database target (`DROP TABLE/VIEW/... IF EXISTS`) sebelum membuat objek baru.
  - **Ignore DEFINER** (Checkbox — Default: Aktif): Menghapus klausa `DEFINER=`user`@`host`` dari pernyataan `CREATE VIEW`, `CREATE PROCEDURE`, `CREATE FUNCTION`, `CREATE TRIGGER`, dan `CREATE EVENT`. Dengan demikian, MySQL target otomatis menetapkan pengguna koneksi saat ini sebagai *definer* dan mencegah kegagalan otorisasi di server *cloud* (SQLyog KB #115).
  - **Copy associated triggers** (Checkbox — Default: Aktif): Mengikat *trigger* secara otomatis ke tabel induknya; mencentang atau menghapus centang sebuah tabel akan otomatis menyertakan atau mengecualikan *trigger* milik tabel tersebut.
  - **Disable Foreign Key & Unique Checks** (Checkbox — Default: Aktif): Menonaktifkan pemeriksaan *foreign key* dan *unique index* sementara pada sesi target agar tabel anak dan tabel induk dapat disalin tanpa terkendala urutan relasi (SQLyog Issue #2315).
- **Kontrol Eksekusi & Tombol Stop Instan**: Saat pengguna menekan tombol **Copy**, tombol tersebut langsung berubah menjadi tombol **Stop**. Jika pengguna menekan **Stop**, `CancellationToken` di Rust memutus aliran *cursor* dan menghentikan pengiriman paket berikutnya dengan aman.

### B. Mekanisme Mesin Eksekusi 4-Tahap di Sisi Rust

Untuk menyamai performa tinggi SQLyog pada jaringan lokal maupun jarak jauh (WAN/SSH), mesin Rust mengeksekusi alur berikut secara asinkron:

| Tahap Eksekusi | Operasi Protokol MySQL (Rust Core) | Detail Teknis & Penanganan Kasus Khusus (Edge Cases) |
|---|---|---|
| **Tahap 1: Persiapan Sesi Target** | Inisialisasi variabel sesi & pembacaan batas paket server tujuan | Menjalankan `SET NAMES utf8mb4; SET FOREIGN_KEY_CHECKS=0; SET UNIQUE_CHECKS=0; SET SQL_MODE='NO_AUTO_VALUE_ON_ZERO';` pada koneksi target. Mesin kemudian membaca `SELECT @@max_allowed_packet;` di server target dan menetapkan batas ukuran *chunk buffer* sebesar 75% dari nilai tersebut (dibatasi antara 1 MB hingga 16 MB). |
| **Tahap 2: DDL Tabel & Sanitasi** | `SHOW CREATE TABLE` ``db`.`tbl`` pada sumber → `CREATE TABLE` pada target | Mengekstrak DDL setiap tabel terpilih. Jika *Drop if exists* dicentang, mengeksekusi `DROP TABLE IF EXISTS` ``tbl`` terlebih dahulu. Untuk tabel dengan kolom *virtual/generated*, mesin mengidentifikasi daftar kolom non-generated agar tidak disertakan dalam daftar kolom `INSERT`. |
| **Tahap 3: Streaming Bulk Packetizer** | Unbuffered `SELECT *` pada sumber → Multi-row `INSERT INTO ... VALUES (...),(...)` pada target | *Task Reader* membuka *streaming cursor* tanpa memuat seluruh tabel ke RAM. *Task Builder* merakit baris ke dalam *buffer* `Vec<u8>` menggunakan sintaks *extended bulk insert* dan melakukan *hex-encoding* (`0x...`) untuk kolom `BLOB/BINARY`. Saat *buffer* mencapai ambang paket, *Task Writer* mengeksekusinya dalam satu kali kirim melalui kanal `mpsc` berkapasitas 4 paket (menjaga pemakaian RAM tetap di bawah 85 MB). |
| **Tahap 4: Trigger, View & Pemulihan** | Pembuatan objek non-tabel pasca-data & pemulihan variabel sesi | **Trigger dibuat paling akhir setelah data tabel selesai disalin** agar *trigger* tidak aktif selama penyisipan data massal. Untuk *Views* yang saling bergantung, mesin menerapkan teknik *Temporary Placeholder Table* seperti `mysqldump`/SQLyog (membuat tabel *dummy* sementara dengan kolom yang sesuai, lalu menggantinya dengan `CREATE OR REPLACE VIEW`). Terakhir, mesin memulihkan `SET FOREIGN_KEY_CHECKS=1; SET UNIQUE_CHECKS=1;`. |

## 5. Spesifikasi Kontrak API FFI (Rust ↔ C#) untuk Modul Data Transfer

Komunikasi antara antarmuka C# dan mesin Rust untuk fitur *Data Transfer* dikendalikan melalui tiga fungsi C-ABI utama yang diekspos oleh `navilite-core` dan diikat secara otomatis oleh `csbindgen`:

- `navilite_transfer_start(config_ptr: *const TransferConfigFfi, progress_cb: extern "C" fn(TransferProgressFfi), user_data: *mut c_void) -> TransferHandle`: Memulai pekerjaan transfer asinkron di dalam *runtime* Tokio dan langsung mengembalikan *handle* ke C# tanpa memblokir *UI thread*.

- `TransferProgressFfi` (Struct C-ABI): Mengirimkan pembaruan ke C# setiap 100 milidetik atau setiap paket selesai, mencakup `current_object_name`, `objects_completed`, `total_objects`, `rows_transferred`, `bytes_transferred`, `throughput_mb_per_sec`, dan `status_code`.
- `navilite_transfer_cancel(handle: TransferHandle) -> i32`: Dipanggil saat tombol **Stop** diklik pada dialog C#; memicu `CancellationToken::cancel()` di Rust untuk menghentikan pembacaan kursor dan membersihkan sesi target secara tertib.

## 6. Target Kebutuhan Non-Fungsional (NFR) & Peta Jalan Pengembangan

Keberhasilan proyek NaviLite diukur berdasarkan metrik efisiensi sumber daya, kecepatan transfer data, serta tahapan pengiriman fitur selama 12 minggu pengembangan:

| Indikator Kinerja / Fase | Target Metrik / Durasi | Kriteria Keberhasilan & Deliverable Utama |
|---|---|---|
| **Cold Startup & Idle Memory** | < 400 ms waktu buka; < 45 MB RAM saat *idle* | Jendela utama siap menerima klik koneksi dalam waktu kurang dari setengah detik; ukuran penginstal *standalone* di bawah 25 MB. |
| **Performa Data Grid** | 60 FPS (< 16 ms/frame); < 65 MB RAM untuk 50.000 baris | Pengguliran (*scrolling*) mulus pada tabel lebar berkat *UI Virtualization* dan pembacaan langsung dari *pinned buffer* FFI. |
| **Performa Data Transfer** | > 85% saturasi *bandwidth* jaringan; < 85 MB RAM puncak | Penyalinan tabel 10 juta baris lintas host berjalan stabil tanpa kenaikan konsumsi RAM seiring bertambahnya ukuran tabel. |
| **Fase 1 (Minggu 1–3)** | Pondasi Rust FFI & Manajemen Koneksi MySQL | Setup *workspace* Cargo + solusi .NET 8 + `csbindgen`; koneksi MySQL TCP/SSL/SSH Tunnel; penyimpanan sandi di OS Keychain. |
| **Fase 2 (Minggu 4–6)** | Navigation Tree & Virtualized Data Grid Editor | Pohon skema *lazy-load*; tampilan Grid/Form dengan paginasi, filter/sort cepat, serta bilah *Batch Editing* (*Apply/Discard*). |
| **Fase 3 (Minggu 7–9)** | Mesin Data Transfer Antar-Koneksi (SQLyog UX) | Dialog tunggal *Copy Database/Table to Different Host*; sanitasi `DEFINER`; *streaming bulk packetizer*; tombol *Stop* instan. |
| **Fase 4 (Minggu 10–12)** | SQL Query Editor, Table Designer & SQL Dump | Editor SQL dengan *autocomplete* skema; *Table Designer* visual; fitur *Streamed SQL Dump/Execute* serta impor/ekspor CSV/JSON. |